Drift-Aware Memory-Bounded Batching for Irregular GPU Pipelines: Online Footprint Feedback for Memory-Bounded Workloads

Lee Si Yuan
Independent Researcher
siyuanlee.cs@gmail.com

Abstract

Memory-bounded batching replaces backend occupancy estimates with explicit resource limits. We propose Drift-Aware Memory-Bounded Batching (DAMB), a controller that could revise per-class batch capacities when measured footprints change. A hard guarantee requires a known conservative footprint bound; an average estimate alone cannot guarantee safety under arbitrary shifts. Our deterministic CPU simulation compares a full-budget policy with a 15% reserve policy over three phases. The reserve policy reaches 1.822 normalized items per time unit versus 1.805 (1.009×), but class footprints and estimates are identical in every phase, so the update never changes an estimate. The difference comes from the reserve and discrete scheduling, not feedback adaptation. No GPU hardware, docking code, or molecular data was used.

1 Introduction

Irregular GPU pipelines spend time in host-side decisions such as grouping inputs, allocating buffers, and submitting work across workers and streams. Accordi et al. (Accordi et al., 2026) study these out-of-kernel decisions in the muDock molecular docking mini-app. Their memory-bounded policy chooses the largest class-specific batch that fits the memory assigned to a worker, avoiding occupancy hints that may differ across backends.

A fixed footprint model is useful when a run has a stable input distribution. Screening workloads can instead change over time. A stale estimate can leave memory unused or lead to an unsafe allocation. This paper studies a bounded feedback loop that observes completed batches and updates the next batch capacity. Our contributions are an online estimate-and-reserve rule, a deterministic CPU simulation, and a proposed drift-triggered controller and GPU evaluation protocol. This is a feasibility study, not a reproduction of the hardware results in (Accordi et al., 2026).

2 Method: Drift-Aware Memory-Bounded Batching

2.1 Online estimate

For each class c, maintain an exponentially smoothed footprint estimate:

$$\hat{m}_t(c) = \alpha m_t^{\text{obs}}(c) + (1-\alpha)\hat{m}_{t-1}(c), \quad (1)$$

where $m_t^{\text{obs}}(c)$ is measured from the latest completed batch and α is a smoothing parameter. The initial estimate can come from a warm-up sample or a declared conservative bound.

2.2 Conservative batch capacity

For worker memory budget M_{worker} and reserve factor $\rho \in (0, 1]$, compute

$$B_t(c) = \max\left(1, \left\lfloor \frac{\rho M_{\text{worker}}}{\hat{m}_t(c)} \right\rfloor\right). \quad (2)$$

The reserve accounts for model error and non-payload allocations and must be calibrated with measured peak allocation. Where a reliable hard upper bound $m_{\max}(c)$ exists, enforce the stronger safety check

$$B_t(c)m_{\max}(c) + M_{\text{overhead}} \le M_{\text{worker}}. \quad (3)$$

An estimate alone cannot guarantee safety under an unbounded distribution shift.

2.3 Drift trigger and controller policy

A production controller should update after a fixed observation window and change capacity only when

$$\frac{|m_{\text{obs}}(c) - \hat{m}(c)|}{\max(\hat{m}(c), \epsilon)} > \delta. \quad (4)$$

Table 1: CPU simulation results.

Policy	Time	Throughput	Relative
Static calibration	3988	1.805	1.000×
Online, 15% reserve	3952	1.822	1.009×

On a trigger, it should lower the next capacity immediately, then raise capacity gradually after sustained lower observations. A known class bound or conservative high percentile should cap the estimate. Without a safe bound, retain the existing safe capacity until enough observations are collected. Controller state is per class and device, avoiding transfer of one backend's calibration to another.

3 Experimental Setup

The deterministic Python simulation uses seed 20261106. Each of three phases contains 2,400 items, with light-item proportions 0.90, 0.25, and 0.90. Light items use one normalized memory unit and one work unit; heavy items use eight memory units and six work units. The worker budget is 96 units, four workers execute batches, and each batch adds two normalized launch-cost units.

The static baseline uses initial class footprint estimates and the full memory budget. The online policy uses the same starting estimates, a 0.85 reserve factor, and 0.5 exponential smoothing after each phase. Class-specific jobs are grouped, then batches are greedily assigned to the least-loaded worker. Simulated time is the maximum worker load for each phase, summed across phases. These choices define a scheduling sensitivity model, not a calibrated model of a GPU or muDock. The experiment is available in `adaptive_batching_experiment.py`; its configuration and results are recorded in `results.json`.

4 Results

The online policy differs by less than 1% in this setup. Its phase throughput is 4.131, 0.860, and 4.131, compared with 4.082, 0.853, and 4.082 for static calibration. Class footprints are constant and the initial estimates already match them, so the workload does not meaningfully exercise estimation drift. Integer batch partitioning and greedy worker assignment can account for the small timing difference. This result does not establish a general benefit or a GPU performance gain.

5 Discussion and Limitations

The experiment has two fixed classes, no transfers, allocation fragmentation, kernel execution, external GPU contention, or measurement noise. Its within-class footprints never drift, so it does not test whether feedback helps. The small difference comes from the reserve and discrete batch scheduling. A reserve based on an average estimate is not a hard safety guarantee; that guarantee requires a known per-class upper bound and explicit accounting for overhead.

A real evaluation should integrate DAMB into muDock or a comparable pipeline, replay time-ordered workloads with distribution shifts, and compare fixed memory-bounded batching, online mean estimates, and high-percentile estimates with a hard safety bound. It should report throughput, peak memory, allocation failures, batch wait time, worker idle time, and per-phase results across repeated runs on each supported backend. No such GPU evaluation is claimed here.

6 Conclusion

We proposed a feedback extension to memory-bounded batching: track per-class footprints and revise future capacities when measurements indicate drift. The CPU simulation does not exercise estimate adaptation; it compares a full-budget policy with a 15% reserve policy while both retain the same accurate estimates. Whether a safe controller can improve throughput under realistic within-class drift without increasing memory failures remains open.

Code and formal check. The `code_attachment.zip` provides the simulation and output. Lean checks the conditional memory-bound lemma in `PaperClaims.batch_memory_safe`; the simulation does not establish its conservative-bound assumption for drifting data.

References

G. Accordi, L. Beltrame, T. Pollinger, J. Domke, D. Gadioli, and G. Palermo. 2026. Performance portability of out-of-kernel optimizations on gpus: A molecular docking case study. *arXiv preprint arXiv:2611.06207*.